



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Q-OP

DESIGN DOCUMENT (DD)
AUTOMATION AND CONTROL ENGINEERING

Group: CW

Team Members:

Özgür Deniz Akbay - ID: 11049499

Mauro Ingenito - ID: 10783278

Nermin Bozkurt - ID: 11048958

Professor: Prof. Matteo Camilli

Tutor: Livia Lestingi

Academic Year: 2025-2026

Contents

Contents	i
1 Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Definitions, Acronyms, Abbreviations	1
1.4 Reference Documents	1
1.5 Document Structure	2
2 Architectural Design	3
2.1 Overview	3
2.2 Component View	3
2.3 Deployment View	5
2.4 Runtime View	6
2.4.1 Active Search Loop Execution	6
2.4.2 Machine Lifecycle Cascading Updates	7
2.5 Data Design	8
2.6 Component Interfaces	9
2.6.1 Routine Booking & Optimization Interface	9
2.6.2 Machine Lifecycle Interface	10
3 User Interface Design	11
3.1 Overview	11
3.2 User Mobile Application	11
3.3 Gym Owner Dashboard	11
4 Requirements Traceability	12
5 Implementation, Integration, and Test Plan	13
5.1 Implementation Strategy	13
5.2 Integration Plan	13
Bibliography	14
A Appendix A	15
List of Figures	16
List of Tables	17

1 | Introduction

1.1. Purpose

This Design Document (DD) translates the functional and non-functional requirements established in the Requirements Analysis and Specification Document (RASD) into a comprehensive technical blueprint for the Q-OP system. It defines the software architecture, component interfaces, and structural designs necessary for developers to implement the dynamic load-balancing and equipment-tracking logic.

1.2. Scope

The technical scope of Q-OP encompasses a multi-tenant, cloud-based web application. It consists of a mobile-responsive interface for gym users, a dedicated dashboard for facility managers, and a centralized backend API. The system handles real-time capacity calculations for abstract *MachineTypes*, manages physical hardware lifecycles, and executes routine optimization algorithms to ensure facility loads remain below the maximum capacity threshold (C_{max}).

1.3. Definitions, Acronyms, Abbreviations

- **API (Application Programming Interface):** A set of rules allowing the client application to communicate with the Node.js server.
- **REST (Representational State Transfer):** The architectural style used for the API, relying on stateless, standard HTTP methods (GET, POST, PUT, DELETE).
- **SPA (Single Page Application):** A web application implementation that loads a single HTML document and dynamically updates the content using JavaScript (via Flutter Web) without requiring full page reloads.
- **RBAC (Role-Based Access Control):** The security mechanism restricting system access based on the user's role (e.g., Gym User vs. Gym Owner).
- **JWT (JSON Web Token):** The standard used for securely transmitting authentication state between the client and the API Gateway.

1.4. Reference Documents

- Requirements Analysis and Specification Document (RASD) for Q-OP, Group CW.
- IEEE Std 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.
- Politecnico di Milano Software Engineering II Project Guidelines (A.Y. 2025-2026).

1.5. Document Structure

- **Chapter 2 - Architectural Design:** Defines the high-level system architecture, component mapping, deployment strategy, and API interfaces.
- **Chapter 3 - User Interface Design:** Details the UX strategy and provides interface mockups for the core user and owner interactions.
- **Chapter 4 - Requirements Traceability:** Maps the technical design components directly to the RASD scenarios and product functions to ensure complete coverage.
- **Chapter 5 - Implementation, Integration, and Test Plan:** Outlines the selected technology stack, development phases, and integration testing methodologies.

2 | Architectural Design

2.1. Overview

The Q-OP system adopts a **Multi-tier Client-Server Architecture** communicating via a **RESTful API**. This structural style separates the presentation tier (user interfaces) from the application tier (business logic and load-balancing math) and the data tier (database).

The architecture consists of three primary layers:

- **Presentation Tier (Client):** Comprises a Responsive Web Application (developed using Flutter Web) that delivers a unified Single Page Application (SPA) experience across both mobile and desktop browsers for users and gym owners.
- **Application Tier (Server):** A centralized Node.js backend handling the core business logic, including the C_{max} capacity calculations, routine optimization algorithms, and automated lifecycle state transitions.
- **Data Tier (Database):** A relational PostgreSQL database ensuring ACID compliance for concurrent booking transactions and enforcing strict data isolation between different gym tenants.

Architectural Justification:

This architecture directly addresses the non-functional requirements established in the RASD. A stateless REST API ensures high **Availability** by allowing server traffic to be easily distributed. The rigid separation of the frontend from the data tier ensures **Scalability** and security, preventing cross-tenant data leakage while offloading the heavy routine optimization algorithms to dedicated backend hardware.

2.2. Component View

The Component View (fig. 2.1) illustrates the modular breakdown of the Q-OP system, defining the specific responsibilities of each software element and their inter-dependencies.

Client Tier Components:

- **Responsive Web Application (Flutter Web):** The overarching Single Page Application (SPA) delivered to the client's browser. By compiling Dart into web-optimized JavaScript, HTML, and CSS, it ensures a fluid, native-like experience across all device form factors (mobile, tablet, and desktop) while maintaining a single, easily maintainable codebase.
- **User Interface Module:** The dedicated presentation logic for gym members. It manages the client-side state for the workout routine builder, the time-slot selection interface, and the interactive rendering of the Capacity Engine's optimization prompts.

- **Owner Dashboard Module:** The administrative interface, secured via Role-Based Access Control (RBAC). It provides facility managers with real-time analytics regarding aggregated demand and offers the control mechanisms necessary to trigger hardware lifecycle state changes (e.g., updating a machine to the **Broken** state).

Application Tier Components (Node.js):

- **API Gateway:** The centralized entry point routing incoming REST/HTTPS requests to the appropriate internal services.
- **Auth Manager:** Handles user and owner authentication, session management, and role-based access control.
- **Booking Engine:** Processes reservation requests, manages the Booking FSM, and triggers cancellation protocols.
- **Capacity & Optimization Engine:** The core analytical module. It retrieves current operational parameters (M), calculates C_{max} , and executes the routine re-ordering and alternative-slot algorithms.
- **Lifecycle Manager:** Receives updates from the Owner Dashboard and manages the physical Machine FSM, directly alerting the Capacity Engine to hardware changes.

Data Tier:

- **PostgreSQL Database:** The relational data store ensuring ACID compliance for concurrent booking transactions.

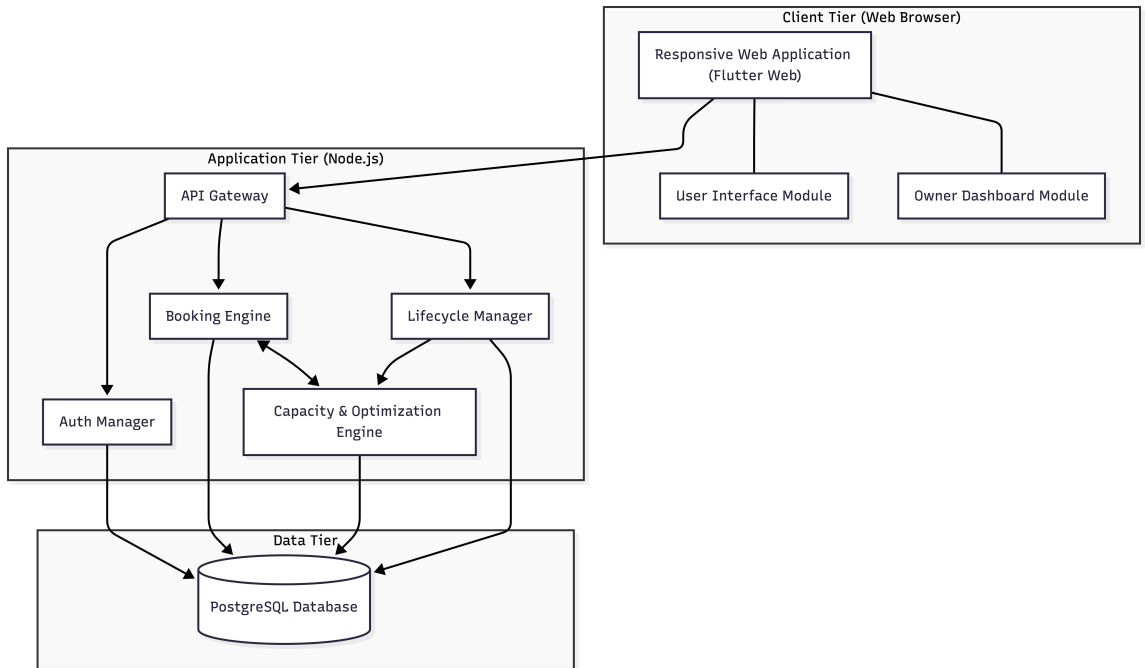


Figure 2.1: Q-OP UML Component Diagram

2.3. Deployment View

The Deployment View (fig. 2.2) defines the physical and network infrastructure required to host the Q-OP system. It illustrates the distribution of software artifacts across distinct hardware nodes, ensuring compliance with the system's non-functional scalability and availability requirements.

Physical Nodes and Network Connections:

- **Client Devices:** Any hardware device (smartphone, tablet, or desktop PC) equipped with a standard web browser. These devices download and execute the Q-OP Web Application artifacts. They communicate with the application server exclusively over secure **HTTPS (REST)** connections to ensure data transit security, granting both users and owners full system access regardless of their physical device.
- **Application Server (Cloud Environment):** A scalable cloud instance (e.g., AWS EC2) hosting the Node.js runtime environment. This node encapsulates the API Gateway, Auth Manager, and the heavy computational logic of the Capacity & Optimization Engine.
- **Database Server:** A separate, managed cloud database node running PostgreSQL. It communicates with the Application Server via a private **TCP/IP** connection within a Virtual Private Cloud (VPC).

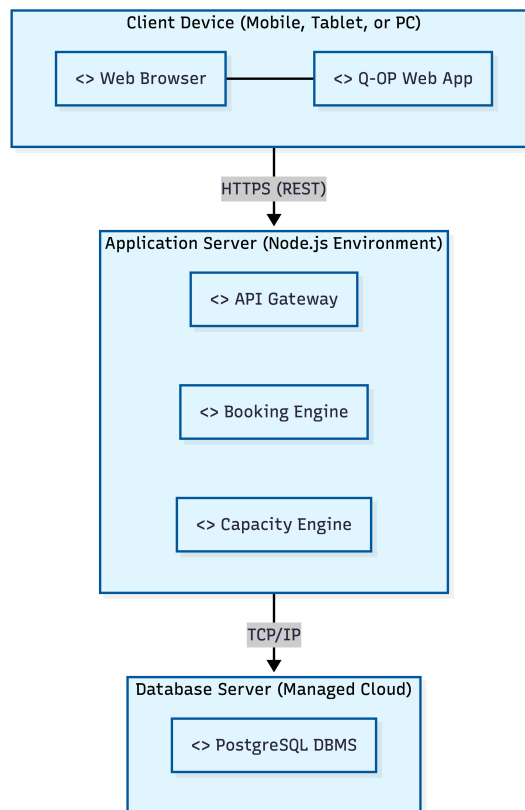


Figure 2.2: Q-OP UML Deployment Diagram

2.4. Runtime View

The Runtime View maps the dynamic interactions between the system's internal components. While the RASD models user-to-system behavior, this section details the intra-component communication required to execute the system's most complex feature: the Active Search Loop for routine optimization.

2.4.1. Active Search Loop Execution

When a user requests a time slot during peak hours, the system does not simply reject the booking if the threshold is exceeded. Instead, the **Capacity & Optimization Engine** actively queries adjacent time slots to resolve the bottleneck automatically (fig. 2.3).

1. **Request Routing:** The *User Interface Module* sends a booking payload to the *API Gateway*, which routes the data to the *Booking Engine*.
2. **Initial Capacity Check:** The *Booking Engine* delegates the validation to the *Capacity Engine*. The engine queries the *PostgreSQL Database* for operational machines (M) and current demand.
3. **Bottleneck Detection:** The *Capacity Engine* calculates C_{max} . If the current demand exceeds this limit, the initial slot is flagged as invalid.
4. **Active Search Loop:** The *Capacity Engine* initiates a localized search algorithm. It queries the database for aggregated demand in adjacent time slots (e.g., ± 30 minutes) and recalculates C_{max} for those specific windows.
5. **Suggestion Generation:** Upon identifying a slot where demand remains below C_{max} , the *Capacity Engine* formulates a suggestion payload. This is passed back through the *API Gateway* to the *User Interface Module*, prompting the user to accept the optimized alternative.

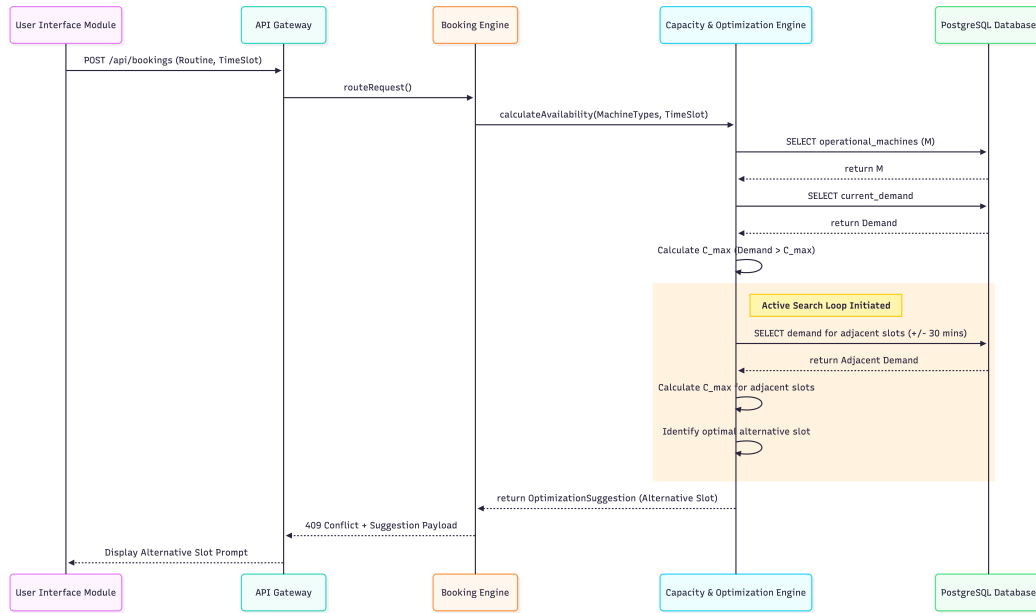


Figure 2.3: Component Interaction: Active Search Loop Optimization

2.4.2. Machine Lifecycle Cascading Updates

When a facility manager updates a physical machine's status to **Broken**, the system must cascade this data across multiple components to ensure the gym does not become overbooked due to the sudden drop in operational capacity (M) (fig. 2.4).

1. **Status Update:** The *Owner Dashboard Module* sends a status update via the *API Gateway* to the *Lifecycle Manager*.
2. **Database Write:** The *Lifecycle Manager* immediately updates the hardware status in the *PostgreSQL Database*, effectively decrementing the available pool (M).
3. **Capacity Review Trigger:** The *Lifecycle Manager* alerts the *Capacity Engine* that a parameter has changed.
4. **Validation and Cancellation:** The *Capacity Engine* recalculates C_{max} for all upcoming slots utilizing that equipment. If the threshold is now breached, it instructs the *Booking Engine* to safely invalidate the excess reservations and notify the affected users.

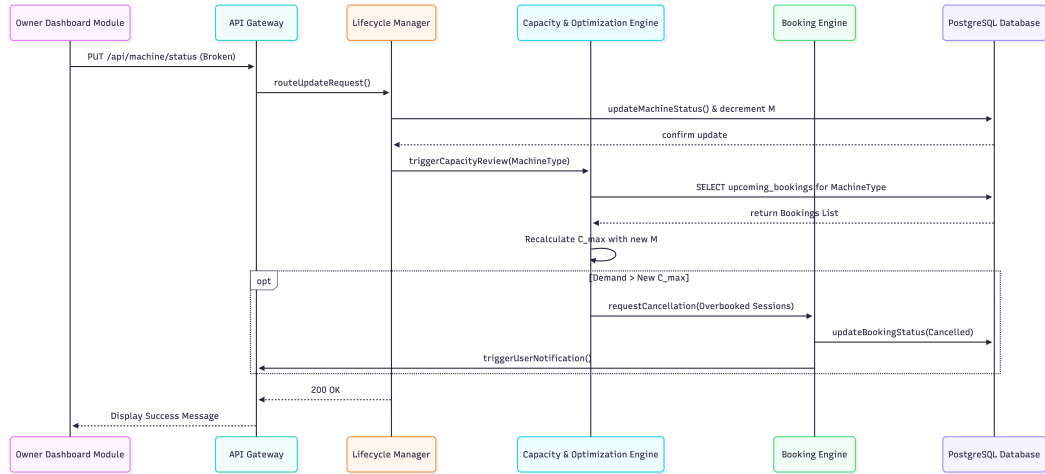


Figure 2.4: Component Interaction: Machine Lifecycle Management

2.5. Data Design

The Data Design section (fig. 2.5) translates the conceptual Domain Model established in the RASD into a Logical Data Model suitable for the PostgreSQL relational database. This schema ensures structural integrity, efficient query execution for the Capacity Engine, and strict data isolation across the multi-tenant architecture.

Core Architectural Data Decisions:

- Normalization & Junction Tables:** To handle the many-to-many relationship between a *WorkoutRoutine* and its required *MachineTypes*, a junction table (`ROUTINE_MACHINE_`) is implemented. This allows the backend to execute highly optimized SQL JOIN queries to instantly aggregate all equipment needed for a specific user session.
- Multi-Tenancy Isolation:** Every physical `MACHINE` is strictly bound to a `gym_id` Foreign Key (FK). When the Capacity Engine queries operational machines (M), it inherently filters by the specific facility, mathematically guaranteeing that load calculations for one gym do not accidentally read data from another.
- ACID Compliance and Locking:** The `BOOKING` table utilizes PostgreSQL's row-level locking mechanisms. During peak hours, when multiple users attempt to book routines requiring the same bottlenecked `MACHINE_TYPE`, the database strictly serializes the transactions. This prevents race conditions and ensures that the total active `status` counts never surpass C_{max} .

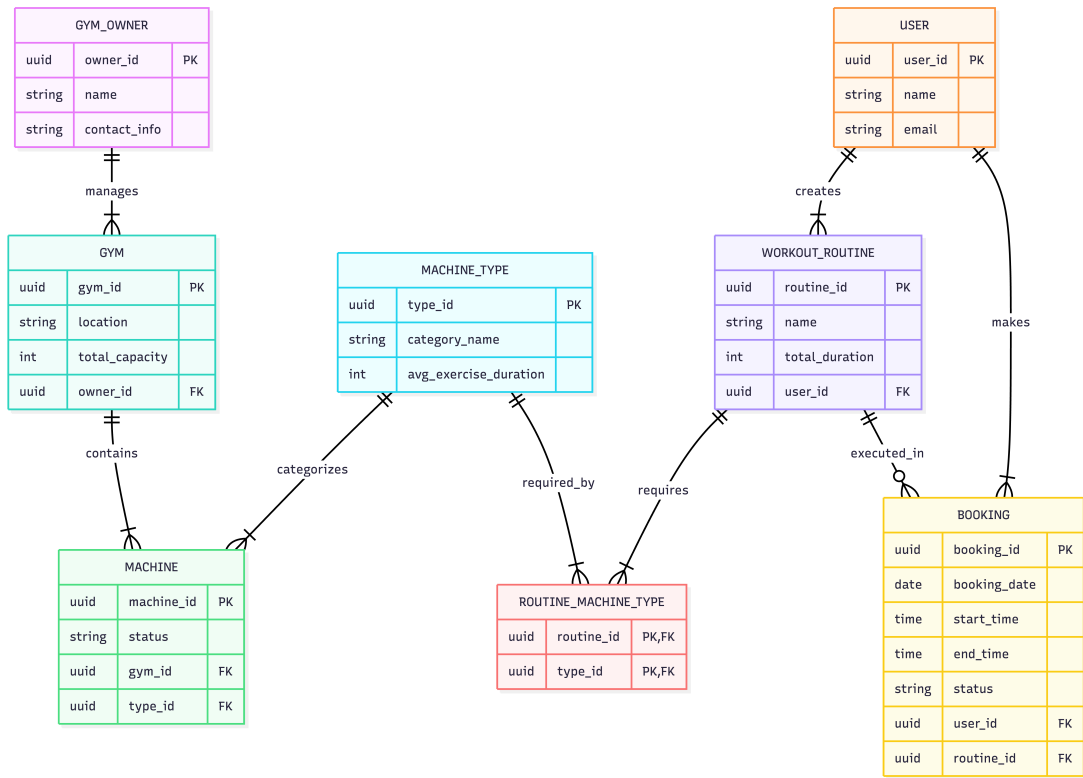


Figure 2.5: Q-OP PostgreSQL Entity-Relationship Diagram (ERD)

2.6. Component Interfaces

The Q-OP system utilizes a RESTful API architecture to facilitate communication between the Client Tier (Flutter Web SPA) and the Application Tier (Node.js). Data is exchanged strictly using lightweight JSON payloads over HTTPS.

Below are the interface contracts for the system's core operational endpoints.

2.6.1. Routine Booking & Optimization Interface

This endpoint handles user requests to schedule a workout. It routes through the Booking Engine and triggers the Capacity Engine's Active Search Loop if bottlenecks are detected.

Endpoint: POST /api/v1/bookings

Request Payload:

```
{
  "user_id": "uuid-1234",
  "routine_id": "uuid-5678",
  "requested_date": "2026-06-25",
  "requested_time": "18:00:00"
}
```

Standard Response (201 Created):

Returned when C_{max} is sufficient and the booking is immediately secured.

```
{
  "status": "success",
  "data": {
    "booking_id": "uuid-9999",
    "state": "Confirmed"
  }
}
```

Optimization Response (409 Conflict):

Returned when the requested slot exceeds C_{max} . The API provides the active search loop's suggested alternative.

```
{
  "status": "optimization_suggested",
  "message": "The requested time slot exceeds maximum facility capacity.",
  "suggestion": {
    "alternative_time": "18:30:00",
    "projected_occupancy_percentage": 85
  }
}
```

2.6.2. Machine Lifecycle Interface

This endpoint allows authorized Gym Owners to update physical hardware states, immediately triggering the Lifecycle Manager to recalculate global capacity.

Endpoint: PUT /api/v1/machines/{machine_id}/status

Request Payload:

```
{
  "owner_id": "uuid-abcd",
  "new_status": "Broken"
}
```

Response (200 OK):

Includes metadata regarding whether the capacity drop resulted in automatic booking cancellations.

```
{
  "status": "success",
  "message": "Machine status updated to Broken.",
  "cascading_effects": {
    "capacity_recalculated": true,
    "bookings_invalidated_count": 3
  }
}
```

3 | User Interface Design

3.1. Overview

3.2. User Mobile Application

3.3. Gym Owner Dashboard

4 | Requirements Traceability

5 | Implementation, Integration, and Test Plan

5.1. Implementation Strategy

5.2. Integration Plan

Bibliography

A | Appendix A

List of Figures

2.1	Q-OP UML Component Diagram	4
2.2	Q-OP UML Deployment Diagram	5
2.3	Component Interaction: Active Search Loop Optimization	7
2.4	Component Interaction: Machine Lifecycle Management	8
2.5	Q-OP PostgreSQL Entity-Relationship Diagram (ERD)	9

List of Tables